



Learnlytics

.handbook

== | **MLOps &**
Model Deployment ==



Concise



Comprehensive



Career-Defining



LEARN SMART
ANALYZE BETTER.
GROW FASTER.



MLOps & Model Deployment

PART 01

A Complete 14-Chapter Solution Manual

First Edition

Published by Learnlytics.handbook

www.learnlyticshandbook.shop | learnlytic.handbook@gmail.com

Copyright © 2026 Learnlytics.handbook. All rights reserved.

No part of this publication may be reproduced, distributed, stored in a retrieval system, or transmitted in any form or by any means — electronic, mechanical, photocopying, recording, or otherwise — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

For permission requests or licensing inquiries, write to the publisher at the address above.

ISBN: 978-1-2565-5895-9 (Digital Edition)

First published: 2026

Printed and distributed digitally worldwide.

IMPORTANT: LICENSE & COPYRIGHT NOTICE This document and all included materials are the exclusive intellectual property of Learnlytics.handbook. Your purchase grants you a single-user, personal license. This file is digitally tracked. Unauthorized reproduction, forwarding, resale, or distribution of this material — whether in private groups, Telegram channels, or public forums — is strictly prohibited and constitutes copyright infringement. Our legal team actively monitors digital distribution networks. Violations of this agreement will result in immediate termination of your access, notification to your educational/professional institution, and formal legal action under applicable international and domestic copyright laws. Thank you for respecting the hard work that went into creating this guide!

Disclaimer: The information in this guide is provided for educational purposes. MLOps tools, cloud platforms, and deployment patterns described here reflect widely accepted industry standards. Always refer to official documentation for the latest API changes and platform-specific configurations. The author and publisher make no warranty, express or implied, with respect to the material contained herein.

Typeset in Arial | Cover design: Learnlytics.handbook

IN Made in India with 💙 for ML Engineers Worldwide

<https://learnlyticshandbook.shop/>

@learnlytics.handbook

TABLE OF CONTENTS

14 Chapters • 280 QS • A Complete MLOps & Deployment Journey

PART 1: From Experiment to Deployment

	CHAPTER TITLE	PAGE
1	Chapter 1: MLflow — Experiment Tracking	04-24
2	Chapter 2: MLflow Model Registry	25-44
3	Chapter 3: DVC — Data Version Control	45-64
4	Chapter 4: Weights & Biases (wandb)	65-88
5	Chapter 5: FastAPI — Model Serving	89-112
6	Chapter 6: Streamlit — Model Demo Apps	113-138
7	Chapter 7: Gradio	139-162
8	Chapter 8: BentoML	163-187
9	Chapter 9: ONNX — Model Portability	188-212
10	Chapter 10: Docker — Containerising Models	213-236
11	Chapter 11: Cloud Run / App Engine	237-260
12	Chapter 12: AWS SageMaker Basics	261-285
13	Chapter 13: Google Vertex AI Basics	286-311
14	Chapter 14: CI/CD for ML (GitHub Actions)	312-339

MLflow — Experiment Tracking

Chapter 1 Test Questions | MLOps & Model Deployment

Level 1: Warmup Test • Level 2: Interviewer Test

LEVEL 1: WARMUP TEST

10 Questions | Build Your Foundational Understanding

SECTION A — Multiple Choice Questions (MCQ) [5 Questions]

Q1 [MCQ]

What is MLflow primarily described as in the context of machine learning workflows?

- A. A deep learning framework for training neural networks
- B. An open-source platform that acts as a 'lab notebook' for ML experiments, recording every run with its configuration and results ✓**
- C. A cloud-based auto-scaling service for model inference
- D. A data preprocessing pipeline tool built by Google

Q2 [MCQ]

Which MLflow component is responsible for storing structured data such as experiment names, run IDs, parameters, and metrics?

- A. Artifact Store
- B. MLflow UI
- C. Backend Store (e.g., SQLite or PostgreSQL) ✓**
- D. Tracking Client SDK

Q3 [MCQ]

Which MLflow function call enables automatic logging of parameters, metrics, and model artifacts without writing any manual `log_*`() calls?

- A. `mlflow.log_all()`

- B. `mlflow.record_run()`
- C. `mlflow.autolog()` ✓**
- D. `mlflow.track_experiment()`

Q4 [MCQ]

What is the key purpose of the `step` argument in `mlflow.log_metric('val_loss', 0.45, step=epoch)`?

- A. It sets the delay (in seconds) before the metric is saved to the server
- B. It specifies which database table to write the metric value into
- C. It records the metric at each epoch so training curves (time-series charts) are visible in the MLflow UI ✓**
- D. It increases the logging precision to avoid floating-point rounding

Q5 [MCQ]

What is the correct command to start an MLflow tracking server using a local SQLite database as the backend store?

- A. `mlflow run --db sqlite:///mlflow.db`
- B. `mlflow server --backend-store-uri sqlite:///mlflow.db --default-artifact-root ./mlartifacts --host 0.0.0.0 --port 5000` ✓**
- C. `mlflow start --backend sqlite --port 5000`
- D. `python -m mlflow.server --uri sqlite --port 5000`

📖 SECTION B — Descriptive Questions [3 Questions]

Q6 [DESCRIPTIVE]

Describe the four core components of MLflow's architecture and explain the role each plays in the experiment tracking lifecycle.

ANS Great question — MLflow's architecture is actually pretty elegant once you break it down. Think of it as four layers working together:

1. **Tracking Client (SDK)** — This lives inside your Python script. When you call `mlflow.log_param()` or `mlflow.log_metric()`, the SDK serialises your data into HTTP payloads and sends them to the server. It's the 'collector' that watches your training code.
2. **Tracking Server** — A REST API server (runs on port 5000 by default). It receives the payloads from the client, validates them, and directs them to the right storage. It also serves the web UI. Think of it as the 'post office' — it routes everything.
3. **Backend Store** — A database (SQLite for dev, PostgreSQL for production) that stores structured, queryable data — params, metrics, tags, and run metadata. This is what the UI reads when you search and filter runs.
4. **Artifact Store** — File storage (local disk, AWS S3, GCS) for large binary files like `model.pkl`, `confusion_matrix.png`, or datasets. Artifacts are uploaded directly by the client (bypassing the server for efficiency), and the server only stores a pointer to them in the Backend Store.

The flow is: Your code → Client → Server → Backend Store (metadata) and Artifact Store (files) → UI reads both and shows you everything. Understanding this split between metadata storage and file storage is key because it determines how you scale — you'd swap SQLite for PostgreSQL and local disk for S3 when going to production.

Q7 [DESCRIPTIVE]

Why does experiment tracking matter in MLOps? Explain at least four concrete consequences of not having it in a production ML team.

ANS **Honestly, experiment tracking is one of those things that seems optional until you've experienced the pain of NOT having it. Here's why it's critical:**

1. **Reproducibility collapses** — Without tracking, after 50 runs you have no record of which learning rate or tree depth produced your 94% accuracy. You literally cannot rebuild your best model from scratch if something goes wrong in production.
2. **Debugging becomes guesswork** — When a deployed model degrades in production, you need to compare its training config against older, better-performing versions. Without logs, there's nothing to compare against.
3. **Team collaboration breaks down** — Two data scientists working on the same problem can silently overwrite each other's best results. There's no audit trail, no shared view of what's been tried.
4. **Compliance failures** — Regulated industries (finance, healthcare, insurance) legally require proof of how a model was built. Without logs, you cannot produce this evidence for regulators or auditors.
5. **Deployment risk** — DevOps and MLOps engineers cannot confidently deploy 'the best model' if there's no systematic record linking a model artifact to its training configuration.

The core insight is that ML development without tracking is like software development without version control — it feels fine on day one and becomes unmanageable by month two.

Q8 [DESCRIPTIVE]

What is the difference between 'parameters' and 'metrics' in MLflow? Why does MLflow treat them differently, and what API calls correspond to each?

**A
N
S**

This distinction is foundational and comes up in almost every MLflow discussion:

Parameters are the inputs YOU set before or at the start of training. They control how the model learns — things like `learning_rate=0.01`, `n_estimators=200`, `batch_size=32`. Parameters are fixed; they don't change during training. You log them with `mlflow.log_param('lr', 0.01)` or `mlflow.log_params({'lr': 0.01, 'epochs': 50})`.

Metrics are the outputs the model produces — they measure how well training is going. Accuracy, F1-score, AUC-ROC, loss, and inference time are all metrics. Crucially, metrics can change over time (loss per epoch), which is why MLflow's `log_metric()` has a `step` argument. You use `mlflow.log_metric('val_loss', 0.42, step=5)` to capture how performance evolves during training.

Why the distinction matters:

- Parameters are searchable inputs — you filter runs by them ('show me all runs where `max_depth < 6`').
- Metrics are comparable outputs — you sort and rank runs by them ('give me the top 10 by AUC-ROC').
- Metrics support time-series (`step`), parameters do not — because a parameter like `batch_size=32` doesn't evolve during training, but `val_loss` changes every epoch.

The mental model: parameters = recipe ingredients (fixed before you start cooking), metrics = how the dish tasted (measured throughout and after).



SECTION C — Coding Questions [2 Questions]

Q9 [CODING]

Write a complete MLflow experiment tracking script that: (1) sets the tracking URI and experiment name, (2) logs at least 4 hyperparameters, (3) trains a `RandomForestClassifier` on the breast cancer dataset, (4) logs 3 evaluation metrics, and (5) logs the trained model as an artifact.

ANS Here's a clean, complete script that covers all five requirements:

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

# (1) Configure MLflow – set where to log and which experiment
mlflow.set_tracking_uri('http://localhost:5000')
mlflow.set_experiment('breast-cancer-classifier')

# Load dataset
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# (2) Define hyperparameters
params = {
    'n_estimators': 150,
    'max_depth': 6,
    'min_samples_split': 5,
    'random_state': 42
}

with mlflow.start_run(run_name='rf_experiment_v1'):

    # Log all hyperparameters at once
    mlflow.log_params(params) # (2) ✓

    # (3) Train the model
    model = RandomForestClassifier(**params)
    model.fit(X_train, y_train)

    # Evaluate
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    # (4) Log evaluation metrics
    mlflow.log_metrics({
        'accuracy': accuracy_score(y_test, y_pred),
```

```
'f1_score': f1_score(y_test, y_pred),
'auc_roc': roc_auc_score(y_test, y_prob)
})

# (5) Log the trained model as a deployable artifact
mlflow.sklearn.log_model(
    sk_model=model,
    artifact_path='model',
    registered_model_name='breast-cancer-rf'
)

print('Run complete! Open MLflow UI at http://localhost:5000')
```

ANS

Key points to remember:

- `log_params()` takes a dict — cleaner than calling `log_param()` five times
- `log_metrics()` similarly accepts a dict for batch logging
- `mlflow.sklearn.log_model()` is preferred over pickle because it adds a model signature, MLmodel metadata, and makes the model directly loadable by deployment tools
- The `with mlflow.start_run()` context manager automatically ends the run cleanly even if an exception occurs

Q10 [CODING]

Enable MLflow auto-logging for scikit-learn, train a LogisticRegression model on the Iris dataset, and add two custom tags and one custom business metric on top of the auto-logged data.

ANS

Auto-logging is powerful — here's how to use it while still adding your own custom context:

```
import mlflow
import mlflow.sklearn
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

mlflow.set_tracking_uri('http://localhost:5000')
mlflow.set_experiment('iris-classifier')

# Enable sklearn auto-logging — logs params, metrics, and model automatically
mlflow.sklearn.autolog(
    log_input_examples=True, # logs sample rows for model signature
```

```

log_model_signatures=True, # captures input/output schema
log_models=True # saves the model artifact
)

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

with mlflow.start_run(run_name='logreg_autolog_demo'):

    # autolog handles all parameter and metric logging for fit()
    model = LogisticRegression(C=0.8, max_iter=300, solver='lbfgs')
    model.fit(X_train, y_train) # <-- autolog fires here

    # Add custom TAGS to categorise this run
    mlflow.set_tag('team', 'data-science-pod-2')
    mlflow.set_tag('data_version', 'iris-v1.0')

    # Add a custom BUSINESS metric on top of auto-logged accuracy
    # (e.g., estimated cost saving from model deployment)
    mlflow.log_metric('estimated_annual_savings_usd', 45000.0)

print('Auto-logging complete — check MLflow UI for full run details')

```

ANS Why this matters:

- autolog() alone gives you C=0.8, max_iter=300, training_score, and the model artifact automatically — zero manual code
- Tags are searchable in the UI: tags.team = 'data-science-pod-2' — great for filtering across experiments
- Custom metrics let you log business KPIs alongside technical metrics — the autolog baseline plus your own context tells a complete story
- **Important: always call mlflow.sklearn.autolog() BEFORE model instantiation, not after — it patches the .fit() method at enable-time**



LEVEL 2: INTERVIEWER TEST

10 Intermediate–Semi-Advanced Questions | Think Like a Senior Practitioner



SECTION A — Scenario-Based MCQ [3 Questions]

Q1 [SCENARIO MCQ]

SCENARIO: A fintech startup has 2 data scientists who started with local MLflow (./mlruns folder). The team has grown to 8 scientists, and now no one can see each other's experiments — everyone has a separate local database. What is the **CORRECT** next step to fix this?

- A. Ask each data scientist to manually copy their mlruns folder to a shared network drive every evening
- B. Deploy a shared MLflow tracking server on a central machine (or EC2), configure a PostgreSQL backend store, set up an S3 artifact store, and have all team members update their `mlflow.set_tracking_uri()` to point to the shared server URL ✓**
- C. Switch from MLflow to Weights & Biases because MLflow does not support multi-user setups
- D. Create a separate Experiment for each data scientist so their runs don't interfere

Q2 [SCENARIO MCQ]

SCENARIO: After 60 training runs over 3 weeks, a data scientist tells their manager 'Run 47 was the best — 93% AUC-ROC.' The manager asks them to redeploy that model. The scientist can't — they have no record of Run 47's hyperparameters, and the model file is gone. Which MLflow features, if used from the start, would have prevented this?

- A. `mlflow.set_experiment()` and `mlflow.set_tag()` only
- B. `mlflow.log_params()` to record hyperparameters AND `mlflow.sklearn.log_model()` (or `log_artifact`) to permanently link the model file to the run — both retrievable via the run's unique `run_id` at any time ✓**
- C. `mlflow.search_runs()` with a `filter_string` — it can reconstruct lost runs from the database
- D. `mlflow.autolog(disable=True)` to prevent accidental overwriting of run data

Q3 [SCENARIO MCQ]

SCENARIO: A healthcare AI team is building a diagnostic model. Regulators require that for any model decision, the team must produce: (a) exact model artifact, (b)

preprocessing pipeline, (c) training data statistics, and (d) validation plots — all from the same training run. Which MLflow approach BEST satisfies this compliance requirement?

- A. Log only the final model using `mlflow.sklearn.log_model()` and keep other files in a separate S3 bucket
- B. Use `mlflow.log_artifact()` individually for the model file only — other files can be stored in git
- C. Log all four items as artifacts in the same MLflow run using `mlflow.sklearn.log_model()` for the full pipeline, `mlflow.log_artifact()` for the data statistics CSV and validation plots — all linked to one `run_id` retrievable at any future date ✓**
- D. Store everything in a relational database and only use MLflow for metric tracking

SECTION B — Scenario-Based Descriptive [2 Questions]

Q4 [SCENARIO DESCRIPTIVE]

SCENARIO: A 3-person ML startup is expanding to 8 data scientists. They currently run mlflow locally (SQLite backend, local artifacts). Describe step-by-step how you would migrate this setup to support the full team, and explain why each step is necessary.

**A
N
S**

This is a classic scaling scenario and the architecture section (1.2) covers it well. Here's how I'd approach it:

Step 1 — Set up a central Tracking Server

Deploy the MLflow server on a shared machine (e.g., AWS EC2 instance). This gives the whole team a single URL to point their experiments at. Without a central server, everyone writes to their own local `mlruns` folder — invisible to teammates.

Step 2 — Migrate from SQLite to PostgreSQL

SQLite is a file-based database — it breaks under concurrent writes from 8 scientists. Switch the `--backend-store-uri` to a PostgreSQL instance (Amazon RDS works well). PostgreSQL handles concurrent reads/writes safely and is queryable at scale.

Step 3 — Move Artifacts to Cloud Storage (S3)

Model files and plots can be large (GBs). Storing them on the tracking server's local disk creates a disk-full risk and means artifacts vanish if the server is restarted. Set `--default-artifact-root s3://company-mlflow-artifacts/` so every team member writes to a durable, shared location.

Step 4 — Update Team Members' Tracking URI

Each scientist adds one line to their scripts or .env file: `MLFLOW_TRACKING_URI=http://<ec2-ip>:5000`. The rest of their code stays identical — this is the beauty of MLflow's modular architecture.

Step 5 — Set IAM / credentials for S3 access

Ensure AWS credentials (via IAM roles or env vars: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`) are available on both the tracking server and each scientist's machine so artifact uploads succeed.

Migration time estimate: < 1 day. The team's experiment code doesn't change at all — only infrastructure and environment variables. This is exactly the startup-to-cloud migration described in the chapter's industry use case.

Q5 [SCENARIO DESCRIPTIVE]

SCENARIO: A hospital AI team needs their experiments to reproduce identically across (1) data scientists' laptops, (2) a GPU training cluster, and (3) a CI/CD pipeline — while remaining HIPAA-compliant. Design the MLflow setup to satisfy these requirements.

A
N
S

This is a multi-environment reproducibility problem with a compliance layer added. Here's how I'd design it:

Core Architecture Decisions:

- Single Shared Tracking Server (EC2/on-prem) — one server means all three environments log to the same Backend Store. Any run is visible to the full team regardless of where it ran.
- PostgreSQL Backend Store — required for concurrent access from laptops + cluster + CI/CD simultaneously. SQLite would corrupt under parallel writes.
- HIPAA-Compliant S3 Artifact Store — use an S3 bucket with: server-side encryption (SSE-S3 or SSE-KMS), versioning enabled, access logging, and a bucket policy restricting access to the team's IAM roles only.

Environment Configuration (the key insight):

Each environment (laptop, cluster, CI/CD) gets a .env file or environment variable injection with:

```
MLFLOW_TRACKING_URI=http://mlflow-server.hospital.internal:5000
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY (or IAM instance roles for cluster/EC2)
MLFLOW_EXPERIMENT_NAME=diagnostic-model-v3
```

Why environment variables over hardcoded URIs:

The experiment code itself has zero environment-specific config. Swap the .env file and the same script runs in all three environments — this guarantees identical tracking behavior.

Compliance-specific additions:

- Log preprocessing pipeline, training data statistics, and validation plots as artifacts so every run is a complete, auditable record
- Use `mlflow.set_tag('phi_data_used', 'false')` or equivalent tags to document data handling
- Set Backend Store on a PostgreSQL instance within the hospital's VPC — never a public endpoint

Result: a regulator asking for 'the model you deployed last March' gets: `run_id` → `params` → `metrics` → `model artifact` → `pipeline artifact` → `validation plots`, all in one click.

**SECTION C — Coding Questions [3 Questions]****Q6 [CODING]**

Write a training loop that logs `train_loss`, `val_loss`, `train_acc`, and `val_acc` at EVERY epoch using the step parameter, so all four metrics appear as time-series charts in the MLflow UI. Use 15 simulated epochs.

ANS The step parameter is what transforms a flat metric into a living training curve. Here's the full implementation:

```
import mlflow
import numpy as np

mlflow.set_tracking_uri('http://localhost:5000')
mlflow.set_experiment('neural-net-training-curves')

NUM_EPOCHS = 15

with mlflow.start_run(run_name='training_curve_demo'):

    # Log fixed hyperparameters once at the start
    mlflow.log_params({
        'num_epochs': NUM_EPOCHS,
        'learning_rate': 0.001,
        'optimizer': 'adam',
        'batch_size': 32
    })

    for epoch in range(NUM_EPOCHS):
```

```
# Simulated training metrics – replace with real model calls
train_loss = 1.0 / (epoch + 1) + np.random.normal(0, 0.02)
val_loss = 1.1 / (epoch + 1) + np.random.normal(0, 0.04)
train_acc = 1.0 - (train_loss * 0.75)
val_acc = 1.0 - (val_loss * 0.75)

# KEY: step=epoch creates the x-axis in MLflow UI charts
mlflow.log_metric('train_loss', round(train_loss, 4), step=epoch)
mlflow.log_metric('val_loss', round(val_loss, 4), step=epoch)
mlflow.log_metric('train_acc', round(train_acc, 4), step=epoch)
mlflow.log_metric('val_acc', round(val_acc, 4), step=epoch)

print(f'Epoch {epoch+1:02d}/{NUM_EPOCHS} | '
      f'val_loss={val_loss:.4f} val_acc={val_acc:.4f}')

print('Training complete – view time-series charts in MLflow UI')

# In the MLflow UI: click the run → Metrics tab
# You will see 4 line charts, one per metric, with epoch on the x-axis
# val_loss rising while train_loss falls = overfitting signal
```

ANS Critical understanding — what step= actually does:

Without step=epoch, MLflow stores only one value per metric name (the last call wins). With step=epoch, it stores 15 (epoch, value) pairs and the UI renders them as a connected line chart. This is how you detect overfitting: if val_loss starts rising at epoch 4 while train_loss still falls, that's your early-stopping signal.

Q7 [CODING]

Write a script that logs a FULL sklearn pipeline (preprocessor + GradientBoostingClassifier) as a single MLflow artifact, then shows how to retrieve and use that pipeline in a separate prediction script.

ANS Logging only the model without the preprocessing pipeline is one of the most common MLOps mistakes — it breaks deployment. Here's the correct approach:

```
# — training_script.py —————
import mlflow
import mlflow.sklearn
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.ensemble import GradientBoostingClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

mlflow.set_tracking_uri('http://localhost:5000')
mlflow.set_experiment('full-pipeline-logging')

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

with mlflow.start_run(run_name='full_pipeline_v1') as run:
    RUN_ID = run.info.run_id # save for retrieval later

    mlflow.log_params({
        'n_estimators': 200,
        'learning_rate': 0.05,
        'scaler': 'StandardScaler'
    })

    # Build the complete pipeline: scaler + model together
    full_pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('classifier', GradientBoostingClassifier(
            n_estimators=200, learning_rate=0.05, random_state=42
        ))
    ])

    full_pipeline.fit(X_train, y_train)

    y_prob = full_pipeline.predict_proba(X_test)[:, 1]
    mlflow.log_metric('auc_roc', roc_auc_score(y_test, y_prob))

    # Log the ENTIRE pipeline - scaler + model in one artifact
    mlflow.sklearn.log_model(
        sk_model=full_pipeline,
        artifact_path='full_pipeline',
        registered_model_name='cancer-detector-pipeline'
    )

    print(f'Pipeline logged. Run ID: {RUN_ID}')

```

— prediction_script.py —————

```

import mlflow.sklearn
import numpy as np

RUN_ID = '<paste-your-run-id-here>'

# Load the complete pipeline — scaler + model, ready to use
pipeline = mlflow.sklearn.load_model(f'runs:{RUN_ID}/full_pipeline')

# New raw data — NO manual scaling needed, pipeline handles it
new_data = np.array([[17.99, 10.38, 122.8, 1001, 0.1184, 0.2776,
    0.3001, 0.1471, 0.2419, 0.07871,
    1.095, 0.9053, 8.589, 153.4, 0.006399,
    0.04904, 0.05373, 0.01587, 0.03003, 0.006193,
    25.38, 17.33, 184.6, 2019, 0.1622, 0.6656,
    0.7119, 0.2654, 0.4601, 0.1189]])

prediction = pipeline.predict(new_data)
probability = pipeline.predict_proba(new_data)[:, 1]

print(f'Prediction: {prediction[0]} (1=malignant, 0=benign)')
print(f'Malignancy probability: {probability[0]:.4f}')

```

ANS Why this approach is production-correct:

- The StandardScaler's fitted mean and variance are baked INTO the logged pipeline — you don't need to re-fit or separately load the scaler at prediction time
- If you only logged the GradientBoostingClassifier, the prediction script would need to manually scale new data with the same scaler — which may not be available or may be a different version
- The registered_model_name parameter adds the pipeline to MLflow's Model Registry, enabling version control and stage transitions (Staging → Production) via the UI

Q8 [CODING]

Write a Python script that uses `mlflow.search_runs()` to find the top 5 runs from a specific experiment filtered by AUC-ROC > 0.88, sorted by AUC-ROC descending, and then programmatically loads and uses the best model for predictions.

ANS This is the programmatic version of what you'd normally do in the UI — essential for CI/CD pipelines that need to auto-select the best model:

```

import mlflow
import mlflow.sklearn
import pandas as pd

```

```

import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

mlflow.set_tracking_uri('http://localhost:5000')

# — Step 1: Search for top runs —————
df = mlflow.search_runs(
    experiment_names=['breast-cancer-classifier'],
    filter_string='metrics.auc_roc > 0.88 AND attributes.status = "FINISHED"',
    order_by=['metrics.auc_roc DESC'],
    max_results=50 # fetch up to 50, then take top 5
)

if df.empty:
    print('No runs found matching the criteria.')
    exit()

# — Step 2: Display top 5 results —————
top5 = df[['run_id', 'metrics.auc_roc', 'metrics.f1_score',
    'params.n_estimators', 'params.max_depth']].head(5)

print('Top 5 Runs by AUC-ROC:')
print(top5.to_string(index=False))
print()

# — Step 3: Get the single best run —————
best_row = df.loc[df['metrics.auc_roc'].idxmax()]
best_run_id = best_row['run_id']
best_auc = best_row['metrics.auc_roc']

print(f'Best Run ID : {best_run_id}')
print(f'Best AUC-ROC: {best_auc:.4f}')
print()

# — Step 4: Load the best model from the artifact store —————
best_model = mlflow.sklearn.load_model(f'runs:{best_run_id}/model')

# — Step 5: Use it for predictions —————
X, y = load_breast_cancer(return_X_y=True)
_, X_test, _, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

predictions = best_model.predict(X_test)
probabilities = best_model.predict_proba(X_test)[: , 1]

```

```
print(f'Sample predictions (first 5): {predictions[:5]}')
print(f'Sample probabilities (first 5): {probabilities[:5].round(3)}')
```

ANS

Interview-worthy insights on this pattern:

- `search_runs()` returns a pandas DataFrame — each row is a run, columns are prefixed: `metrics.<name>`, `params.<name>`, `tags.<name>`
- The `filter_string` uses MLflow Search DSL — similar to SQL WHERE: AND/OR supported, metrics use `>`, `<`, `=`, params/tags use `=` only
- This exact pattern is used in CI/CD pipelines: after a nightly training job completes, a pipeline script calls `search_runs()`, loads the best model, runs integration tests, and if tests pass, registers it in the Model Registry for production
- Prefer `order_by=['metrics.auc_roc DESC']` over pandas sorting — it leverages database-level sorting for efficiency on large experiment databases



SECTION D — Database Scenario-Based Coding [2 Questions]

Q9 [DB CODING]

SCENARIO: Your team is moving MLflow to production. Write the complete setup — including the Docker Compose configuration for a PostgreSQL backend store and the mlflow server startup command — so the entire team can share a single, persistent tracking server.

ANS

This is the production setup pattern from section 1.3 expanded with explanation:

```
# — docker-compose.yml —————
version: '3.8'

services:
  # Service 1: PostgreSQL database as the Backend Store
  mlflow-db:
    image: postgres:14
    container_name: mlflow_postgres
    restart: always
    environment:
      POSTGRES_DB: mlflow
      POSTGRES_USER: mlflow_user
      POSTGRES_PASSWORD: mlflow_secure_pass_2024
    volumes:
      - postgres_data:/var/lib/postgresql/data # persist DB on disk
```

```

ports:
- '5432:5432'
healthcheck:
test: ['CMD', 'pg_isready', '-U', 'mlflow_user', '-d', 'mlflow']
interval: 10s
retries: 5

# Service 2: MLflow Tracking Server
mlflow-server:
image: ghcr.io/mlflow/mlflow:latest
container_name: mlflow_server
restart: always
ports:
- '5000:5000' # UI + REST API
environment:
# Backend Store: PostgreSQL connection string
MLFLOW_BACKEND_STORE_URI: >-
postgresql://mlflow_user:mlflow_secure_pass_2024
@mlflow-db:5432/mlflow
# Artifact Store: shared local volume (swap for S3 URI in production)
MLFLOW_DEFAULT_ARTIFACT_ROOT: /mlflow/artifacts
volumes:
- mlflow_artifacts:/mlflow/artifacts
depends_on:
mlflow-db:
condition: service_healthy # wait for DB to be ready
command: >
mlflow server
--backend-store-uri postgresql://mlflow_user:mlflow_secure_pass_2024@mlflow-
db:5432/mlflow
--default-artifact-root /mlflow/artifacts
--host 0.0.0.0
--port 5000

volumes:
postgres_data: # persists even if container restarts
mlflow_artifacts: # persists model files and plots

```

```

# — How to start the stack —————
# Start both services
docker-compose up -d

# Verify both containers are running
docker-compose ps

# Check MLflow server logs

```

```
docker-compose logs mlflow-server

# — How team members configure their scripts —————
# Each team member adds to their .env or shell profile:
export MLFLOW_TRACKING_URI='http://<server-ip>:5000'
export MLFLOW_EXPERIMENT_NAME='shared-team-project'

# Their Python code needs zero changes — just:
import mlflow
mlflow.set_experiment('shared-team-project')
with mlflow.start_run():
    mlflow.log_params({'lr': 0.01, 'epochs': 50})
# ... train ... log metrics ...
```

ANS Why PostgreSQL vs SQLite matters at this scale:

- SQLite is a single-writer database — with 8 scientists running concurrent experiments, writes would block or corrupt each other. PostgreSQL handles full ACID concurrent transactions.
- The healthcheck + depends_on pattern ensures MLflow server doesn't start before the database is accepting connections — a common startup race condition bug
- For true production, replace /mlflow/artifacts volume with --default-artifact-root s3://company-bucket/mlflow — the docker-compose structure stays identical, only the URI changes

Q10 [DB CODING]

SCENARIO: Your MLflow PostgreSQL backend has 300+ runs across 5 experiments. Write Python code that: (1) queries all runs where `f1_score > 0.85` AND `n_estimators > 100`, (2) groups results by experiment name and shows count + avg AUC per group, and (3) exports the filtered results to a CSV report.

ANS This is a real-world data analysis task you'd do before a model review meeting. Here's the complete solution:

```
import mlflow
import pandas as pd
from datetime import datetime

mlflow.set_tracking_uri('http://localhost:5000')

# — Step 1: Query ALL finished runs across ALL experiments —————
# mlflow.search_runs() can search across multiple experiments at once
df = mlflow.search_runs(
    # Search all 5 experiments by name
```

```

experiment_names=[
    'breast-cancer-classifier',
    'credit-risk-model',
    'fraud-detector',
    'churn-predictor',
    'loan-approver'
],
# Filter: f1_score > 0.85 AND n_estimators parameter > 100
# Note: params are strings in MLflow – cast with CAST for numeric compare
filter_string=(
    'metrics.f1_score > 0.85 '
    'AND params.n_estimators > "100" '
    'AND attributes.status = "FINISHED"'
),
order_by=['metrics.auc_roc DESC'],
max_results=500
)

print(f'Runs matching criteria: {len(df)}')
print()

if df.empty:
    print('No runs matched the filter criteria.')
    exit()

# — Step 2: Add experiment name column —————
# Get experiment names from IDs using MLflow client
client = mlflow.MlflowClient()

exp_id_to_name = {}
for exp_id in df['experiment_id'].unique():
    exp = client.get_experiment(exp_id)
    exp_id_to_name[exp_id] = exp.name

df['experiment_name'] = df['experiment_id'].map(exp_id_to_name)

# — Step 3: Group by experiment and compute summary stats —————
summary = df.groupby('experiment_name').agg(
    run_count=('run_id', 'count'),
    avg_auc_roc=('metrics.auc_roc', 'mean'),
    max_auc_roc=('metrics.auc_roc', 'max'),
    avg_f1_score=('metrics.f1_score', 'mean'),
    best_run_id=('run_id', 'first') # first row = highest AUC (sorted DESC)
).reset_index()

summary['avg_auc_roc'] = summary['avg_auc_roc'].round(4)

```

```

summary['max_auc_roc'] = summary['max_auc_roc'].round(4)
summary['avg_f1_score'] = summary['avg_f1_score'].round(4)

print('Summary by Experiment (runs with f1>0.85 & n_estimators>100):')
print(summary.to_string(index=False))
print()

# — Step 4: Export full filtered results to CSV —————
output_cols = [
    'experiment_name', 'run_id', 'start_time',
    'metrics.f1_score', 'metrics.auc_roc', 'metrics.accuracy',
    'params.n_estimators', 'params.max_depth', 'params.learning_rate'
]

# Only keep columns that actually exist in the DataFrame
available_cols = [c for c in output_cols if c in df.columns]
export_df = df[available_cols].copy()

timestamp = datetime.now().strftime('%Y%m%d_%H%M')
filename = f'mlflow_run_report_{timestamp}.csv'
export_df.to_csv(filename, index=False)

print(f'Report exported: {filename} ({len(export_df)} rows)')

```

ANS

Key DB-level insights for the interview:

- `mlflow.search_runs()` translates the `filter_string` into SQL queries against the PostgreSQL Backend Store — it does NOT load all 300 runs into memory first. The filtering happens at the database level, which is why a proper DB backend (not SQLite) matters at scale.
- `params` are stored as strings in the Backend Store even when their values are numbers. Comparisons like `params.n_estimators > '100'` work via string comparison — for strict numeric range queries on `params`, it's safer to filter in pandas after retrieval.
- The `MlflowClient()` pattern gives you low-level access to the tracking server's REST API — you can create experiments, delete runs, transition model stages, and more programmatically, beyond what the high-level `mlflow.*` functions expose.
- *This report generation pattern is commonly used in weekly model review pipelines: triggered automatically by a cron job, exported to CSV, and posted to Slack or emailed to the team before the review meeting.*



QUICK REFERENCE — Chapter 1 Key APIs

MLflow API Call	What It Does	When to Use
<code>mlflow.set_tracking_uri()</code>	Points SDK to tracking server	Top of every script
<code>mlflow.set_experiment()</code>	Creates/selects experiment	Before <code>start_run()</code>
<code>mlflow.start_run()</code>	Opens a new tracked run	Wrap training block
<code>mlflow.log_params(dict)</code>	Logs hyperparameters	After <code>start_run()</code>
<code>mlflow.log_metrics(dict)</code>	Logs output scores	After <code>model.fit()</code>
<code>mlflow.log_metric(n, v, step=)</code>	Logs per-epoch metrics	Inside training loop
<code>mlflow.log_artifact(path)</code>	Saves any file to run	Plots, configs, pipelines
<code>mlflow.sklearn.log_model()</code>	Logs deployable model	Always with full pipeline
<code>mlflow.autolog()</code>	Zero-code auto tracking	Quick prototyping
<code>mlflow.search_runs()</code>	Programmatic run query	CI/CD model selection